

# Plan V2 : Moteur de Recommandation IA – La Foir’Fouille (avec MetaRank)

## Contexte

- **Réalisé par** : 1 stagiaire seul, profil Python / Data Science
- **Durée** : 3 mois (~12 semaines)
- **Livrable** : Prototype intégré proche production (SAP OCC Python Next.js)
- **Données** : ~189k commandes historiques dans MSSQL, CPU only
- **Infra** : PostgreSQL + Redis + MetaRank installés sur WSL Ubuntu (apt install, pas de Docker)

## TL;DR

3 cas d'usage, 3 techniques de génération de candidats, 1 re-ranker MetaRank, tracking temps réel via SAP OCC.

- **Candidats bruts** : SQL time-decay (top ventes) + LightFM (cross-sell) + Sentence-BERT (similarité) stockés dans Redis (même pour tous les users)
- **Personnalisation** : MetaRank re-rank ces candidats en temps réel (~10ms) pour chaque user connecté
- **Tracking** : chaque click/ajout panier/achat est envoyé à MetaRank via un `rankingId` (closed-loop)
- **SAP Commerce** : proxy entre le frontend et le microservice Python (le frontend ne parle jamais directement à Python)

## Architecture Production-Ready

```
graph TD
    Frontend[Next.js Frontend  
(REST/JSON via OCC)]
    SAP[SAP Commerce OCC RecommendationController (nouveau)]
    Facade[RecommendationFoirFacade FastAPI (orchestrateur)]
    Populators[Populators PostgreSQL Redis MetaRank (localhost:8013)]

    Frontend --> SAP
    SAP --> Facade
    Facade --> Populators
```

(HTTP localhost:8000)

(events, candidats (re-rank personnalisé  
produits, bruts pré- temps réel ~10ms)  
historique) calculés,  
même pour tous)

## Flux pour chaque requête :

1. Next.js appelle SAP OCC `GET /occ/v2/{site}/recommendations/...`
2. SAP OCC appelle FastAPI `GET /api/recommendations/...`
3. FastAPI récupère les **candidats bruts** depuis Redis (même liste pour tous)
4. FastAPI envoie les candidats à MetaRank `POST /api/v1/rank` pour **re-ranking personnalisé**
5. FastAPI retourne la liste re-rangée + `rankingId` à SAP
6. SAP convertit les product codes en `ProductData` via populators
7. SAP renvoie `RecommendationWsFoirDTO` au frontend (avec `rankingId` )

8. Quand le user clique frontend envoie `POST /events` avec le `rankingId`

## Pourquoi PostgreSQL ?

- 189k commandes = ~500k-1M interactions SQLite trop lent en écriture concurrente
- Window functions, array\_agg, lateral joins pour les requêtes analytiques
- Index GIN pour JSONB et arrays
- Plus proche d'une architecture production réelle

## Pourquoi Redis ?

- Cache des candidats bruts pré-calculés (même pour tous) latence < 5ms
- `topsales:global` , `crosssell:{code}` , `similar:{code}`
- **PAS de cache par user** — la personnalisation est faite en temps réel par MetaRank

## Pourquoi MetaRank ?

- Re-ranking personnalisé en **temps réel** (~5-10ms par requête)
- Apprend automatiquement des interactions via LambdaMART/XGBoost
- Features item + features session (clicks récents)
- Remplace le scoring hybride manuel ( $\alpha$ ,  $\beta$ ,  $\gamma$ ,  $\delta$ ,  $\epsilon$ )
- Modèle entraîné en batch (1x/jour), **inférence** en temps réel

## Installation WSL

```
# PostgreSQL + Redis
sudo apt update && sudo apt install postgresql redis-server
sudo service postgresql start
sudo service redis-server start

# MetaRank (JAR Java)
wget https://github.com/metarank/metarank/releases/download/v0.7.9/metarank-assembly-0.7.9.jar
# Lancement : java -jar metarank-assembly-0.7.9.jar standalone --config config.yml --data events.jsonl
```

## Les 3 cas d'usage

| Cas d'usage                | Page                  | Candidats (Redis)                       | Re-ranking (MetaRank)           |
|----------------------------|-----------------------|-----------------------------------------|---------------------------------|
| <b>Top ventes perso</b>    | Accueil               | SQL time-decay top 50 global            | MetaRank re-rank top 12 affiché |
| <b>Cross-sell</b>          | Page produit + Panier | LightFM predict top 20 par produit      | MetaRank re-rank top 6 affiché  |
| <b>Produits similaires</b> | Page produit          | Sentence-BERT cosine top 20 par produit | MetaRank re-rank top 8 affiché  |

## Placements frontend :

- **Page accueil (connecté)** : "Nos recommandations pour vous" top ventes re-ranké par MetaRank
- **Page accueil (anonyme)** : "Meilleures ventes" top ventes brut (pas de re-rank)
- **Page produit** : "Produits similaires" (Sentence-BERT) + "Souvent achetés ensemble" (LightFM cross-sell)
- **Page panier** : "Complétez votre commande" (cross-sell agrégé des items du panier)

## Choix des modèles ML (optimisés CPU, 189k commandes)

| Composant                        | Rôle                    | Pourquoi ce choix                                      | Temps CPU  |
|----------------------------------|-------------------------|--------------------------------------------------------|------------|
| SQL time-decay                   | Candidats top ventes    | Pas de ML, agrégation simple, cron 1x/jour             | Instantané |
| LightFM (WARP)                   | Candidats cross-sell    | Hybrid CF, gère cold start, CPU-friendly               | ~5 min     |
| Sentence-BERT (all-MiniLM-L6-v2) | Candidats similarité    | Sémantique > TF-IDF, 80MB, rapide CPU                  | ~15 min    |
| MetaRank (LambdaMART/XGBoost)    | Re-ranking personnalisé | Temps réel, apprend des clicks/achats, features riches | ~2 min     |

### Pourquoi LightFM pour le cross-sell ?

- **Hybrid** : combine collaborative filtering + content-based → gère le cold start
- **WARP loss** : optimise directement le ranking
- **CPU-friendly** : entraînement en < 10 min sur 189k commandes
- **Compétence ML à valoriser en soutenance** (pas juste “j’ai branché un outil”)
- Alternatives écartées : Deep Learning (trop lourde CPU), SVD/NMF (pas hybrid), Item2Vec (LightFM fait mieux)

### Pourquoi Sentence-BERT pour la similarité ?

- TF-IDF = similarité de mots → “lampe de bureau” ≠ “luminaire de travail”
- Sentence-BERT = similarité sémantique → comprend que c’est similaire
- Modèle **all-MiniLM-L6-v2** : 80MB, rapide CPU, bon pour le français
- Pré-calculer les embeddings une seule fois → stocker dans Redis

### Pourquoi MetaRank pour le re-ranking ?

- Remplace le scoring hybride manuel ( $\alpha \times \text{popularity} + \beta \times \text{affinity} + \dots$ )
- Apprend automatiquement les poids optimaux à partir des interactions réelles
- Features temps réel : clicks de la session en cours → re-rank adapté
- Latence ~5-10ms → transparent pour l'utilisateur

## Phase 1 : Infra & Données (Sem. 1-3)

### Sem. 1 — Setup infra + export MSSQL → PostgreSQL

#### 1. Installer PostgreSQL + Redis + MetaRank sur WSL

#### 2. Créer la base PostgreSQL `lff_recommendations` :

```
CREATE TABLE products (
  product_code VARCHAR(50) PRIMARY KEY,
  name TEXT,
  description TEXT,
  category_codes TEXT[],    -- Array de codes catégorie
  price DECIMAL(10,2),
  season_tags TEXT[],      -- ['NOEL', 'ETE', ...]
  created_at TIMESTAMP,
  embedding FLOAT8[]       -- Sentence-BERT embedding (384 dims)
);

CREATE TABLE user_events (
  id BIGSERIAL PRIMARY KEY,
  user_id VARCHAR(100),
  event_type VARCHAR(20),  -- click, add_to_cart, purchase, wishlist
```

```

product_code VARCHAR(50),
session_id VARCHAR(100),
timestamp TIMESTAMP DEFAULT NOW(),
metadata JSONB
);

CREATE INDEX idx_events_user_time ON user_events(user_id, timestamp DESC);
CREATE INDEX idx_events_product ON user_events(product_code, event_type);
CREATE INDEX idx_products_categories ON products USING GIN(category_codes);

```

### 3. Exporter MSSQL CSV PostgreSQL :

- Via HAC FlexibleSearch : `SELECT {pk}, {code}, {name}, ... FROM {Product}` products.csv
- `SELECT {o.code}, {o.user}, {oe.product}, {oe.quantity}, {oe.basePrice}, {o.date} FROM {Order AS o JOIN OrderEntry AS oe}` orders.csv
- `SELECT {uid}, {defaultShipmentAddress}, {name} FROM {Customer}` customers.csv
- Script Python `scripts/load_csv_to_postgres.py` avec `psycopg2` + `COPY`

## Sem. 2 — Convertir commandes historiques en événements MetaRank

**PAS de simulation Selenium.** Conversion directe des commandes en format MetaRank JSONL.

### 4. Script `scripts/convert_orders_to_events.py` :

```

# Pour chaque produit unique item event (métadonnées)
{"event":"item","id":"LAMPE-001","timestamp":"2025-01-01T00:00:00Z",
"fields":[
{"name":"category","value":"decoration"},
{"name":"price","value":29.99},
{"name":"title","value":"Lampe de bureau LED"}
]}

# Pour chaque commande ranking event (produits achetés + négatifs aléatoires)
{"event":"ranking","id":"hist-order-12345","timestamp":"2025-03-15T14:00:00Z",
"user":"user123","session":"hist-sess-12345",
"items":[
{"id":"LAMPE-001"}, {"id":"VASE-002"}, {"id":"COUSSIN-003"},
{"id":"TABLE-004"}, {"id":"BOUGIE-005"}
]}

# Pour chaque produit acheté dans la commande interaction event
{"event":"interaction","id":"int-12345","timestamp":"2025-03-15T14:01:00Z",
"ranking":"hist-order-12345","item":"LAMPE-001","type":"purchase"}

```

### Logique pour construire les ranking events historiques :

- Pour chaque commande : produits achetés + N produits aléatoires de la même catégorie (négatifs)
- Ratio : ~3 négatifs pour 1 positif
- Résultat : ~189k commandes × ~5 items = ~1M events JSONL

**5. Convertir aussi en events PostgreSQL** (INSERT INTO user\_events) pour le calcul des top ventes SQL et l'entraînement LightFM.

## Sem. 3 — Config MetaRank + EDA

### 6. Fichier de config MetaRank `metarank/config.yml` :

```

models:
default:

```

```

type: lambdamart
backend:
  type: xgboost
features:
  - popularity
  - category_match
  - price_bucket
  - item_ctr
  - user_click_count

features:
  - name: popularity
    type: number
    scope: item
    source: item.popularity

  - name: category_match
    type: boolean
    scope: item
    # Catégorie du produit match les catégories préférées du user

  - name: price_bucket
    type: number
    scope: item
    source: item.price
    buckets: [5, 10, 20, 50, 100, 200]

  - name: item_ctr
    type: rate
    top: click
    bottom: impression
    scope: item
    bucket: 24h

  - name: user_click_count
    type: interacted_with
    interaction: click
    field: item.category
    scope: session

```

#### 7. Entraîner MetaRank sur les données historiques :

```

java -jar metarank-assembly-0.7.9.jar standalone \
  --config metarank/config.yml \
  --data events.jsonl

```

#### 8. Notebook EDA `notebooks/eda.ipynb` :

- Distribution des produits par catégorie
- Matrice de co-achat (produits achetés ensemble dans la même commande)
- Saisonnalité : volume de ventes par mois et par catégorie
- Top produits / top catégories

## Phase 2 : Modèles ML + Candidats (Sem. 4-6)

### Sem. 4 — Top ventes (SQL) + Sentence-BERT (similarité)

## 9. Top ventes global avec time-decay (SQL) :

```
SELECT product_code,
       SUM(EXP(-0.01 * EXTRACT(DAY FROM NOW() - timestamp))) AS score
FROM user_events
WHERE event_type = 'purchase'
      AND timestamp > NOW() - INTERVAL '90 days'
GROUP BY product_code
ORDER BY score DESC
LIMIT 50;
```

Résultat stocké dans Redis : `topsales:global = [A, B, C, ...]`

CronJob : recalculer 1x/jour via `scripts/precompute_cache.py`

## 10. Sentence-BERT pour similarité produit :

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')

# Encoder tous les produits
for product in products:
    text = f'{product.name} {' '.join(product.categories)}'
    embedding = model.encode(text)
    # Stocker dans PostgreSQL (products.embedding) et Redis
```

- Pré-calculer top-20 similaires pour chaque produit    Redis : `similar:{product_code}`
- Cosine similarity entre embeddings

## Sem. 5 — LightFM (cross-sell)

### 11. LightFM pour cross-sell :

```
from lightfm import LightFM
from lightfm.data import Dataset

# Construire la matrice d'interactions
dataset = Dataset()
dataset.fit(users, items, item_features=item_feature_labels)

interactions, weights = dataset.build_interactions(
    [(user, item, weight) for user, item, weight in events]
    # weight: view=1, wishlist=2, add_to_cart=3, purchase=5
)

item_features = dataset.build_item_features(
    [(item, [cat, season, price_bucket]) for item in items]
)

# Entraîner
model = LightFM(loss='warp', no_components=64, learning_rate=0.05)
model.fit(interactions, item_features=item_features, epochs=30, num_threads=4)

# Pré-calculer cross-sell pour chaque produit
for product_idx in range(n_products):
    scores = model.predict(user_ids=0, item_ids=all_items, item_features=item_features)
    top_20 = np.argsort(-scores)[:20]
    redis.set(f'crosssell:{product_code}', json.dumps(top_20_codes))
```

Redis : `crosssell:{product_code} = [P1, P2, P3, ...]`

## Sem. 6 — Évaluation + pipeline batch

### 12. Évaluation offline :

- Split temporel : train = commandes mois 1 à M-1, test = mois M
- Métriques :
  - **Precision@10** : % de produits recommandés réellement achetés
  - **Recall@10** : % de produits achetés qui étaient recommandés
  - **NDCG@10** : qualité du ranking
  - **Coverage** : % du catalogue dans les recommandations
  - **Diversity** : nb catégories distinctes dans le top-10
- Comparer : Top ventes brut vs LightFM vs Sentence-BERT vs avec MetaRank re-rank
- Target : Precision@10 > 5%, NDCG > 0.1 (réaliste sur du retail)

### 13. Script batch `scripts/precompute_cache.py` :

```
# Exécuté 1x/jour (cron ou manuellement)
# 1. Recalculer top ventes global   Redis topsales:global
# 2. Re-scoring LightFM cross-sell pour produits actifs   Redis crosssell:{code}
# 3. Recalculer similarité top-20   Redis similar:{code}
# 4. Mettre à jour les item events MetaRank (nouveaux produits)
```

## Phase 3 : API FastAPI + MetaRank + Tracking (Sem. 7-8)

### Sem. 7 — FastAPI endpoints + client MetaRank

#### 14. Structure du microservice :

```
recommendation-engine/
  main.py           # FastAPI app
  config.py         # Settings (URLs, paths, weights)
  routers/
    recommendations.py # Endpoints recommandations
    events.py         # Endpoint tracking
  services/
    metarank_client.py # Client HTTP MetaRank
    candidate_service.py # Récupération candidats Redis
    tracking_service.py # Enregistrement events
  data/
    postgres_client.py # Connexion PostgreSQL (asyncpg)
    redis_client.py    # Connexion Redis
  scripts/
    convert_orders_to_events.py # Commandes   MetaRank JSONL
    export_sap_data.py         # FlexibleSearch   CSV
    load_csv_to_postgres.py    # CSV   PostgreSQL
    train_lightfm.py          # Entraînement LightFM
    compute_embeddings.py      # Sentence-BERT embeddings
    precompute_cache.py       # Pré-calcul candidats   Redis
  metarank/
    config.yml          # Config MetaRank
  notebooks/
    eda.ipynb
    evaluation.ipynb
  tests/
    test_recommendations.py
```

requirements.txt  
README.md

## 15. Endpoints FastAPI :

```
# =====
# GET /api/recommendations/top-sales?user_id=xxx&limit=12
# Page accueil — Top ventes re-ranké si user connecté
# =====
@router.get("/top-sales")
async def top_sales(user_id: str = None, limit: int = 12):
    # 1. Candidats bruts depuis Redis (même pour tous)
    candidates = await redis.get("topsals:global") # [A, B, C, D, ...]

    # 2. Si user connecté re-rank via MetaRank
    ranking_id = None
    if user_id:
        result = await metarank_client.rank(user_id=user_id, items=candidates)
        candidates = result["items"]
        ranking_id = result["ranking_id"]

    return {
        "items": candidates[:limit],
        "type": "top-sales",
        "personalized": bool(user_id),
        "rankingId": ranking_id
    }

# =====
# GET /api/recommendations/cross-sell/{product_code}?user_id=xxx&limit=6
# Page produit + Panier — Cross-sell (LightFM candidats)
# =====
@router.get("/cross-sell/{product_code}")
async def cross_sell(product_code: str, user_id: str = None, limit: int = 6):
    # 1. Candidats LightFM depuis Redis
    candidates = await redis.get(f"crosssell:{product_code}")

    # 2. Re-rank via MetaRank si user connecté
    ranking_id = None
    if user_id:
        result = await metarank_client.rank(user_id=user_id, items=candidates)
        candidates = result["items"]
        ranking_id = result["ranking_id"]

    return {"items": candidates[:limit], "type": "cross-sell", "rankingId": ranking_id}

# =====
# GET /api/recommendations/similar/{product_code}?user_id=xxx&limit=8
# Page produit — Produits similaires (Sentence-BERT candidats)
# =====
@router.get("/similar/{product_code}")
async def similar(product_code: str, user_id: str = None, limit: int = 8):
    # 1. Candidats Sentence-BERT depuis Redis
    candidates = await redis.get(f"similar:{product_code}")

    # 2. Re-rank via MetaRank si user connecté
```

```

ranking_id = None
if user_id:
    result = await metarank_client.rank(user_id=user_id, items=candidates)
    candidates = result["items"]
    ranking_id = result["ranking_id"]

return {"items": candidates[:limit], "type": "similar", "rankingId": ranking_id}

# =====
# GET /api/recommendations/cart?items=P1,P2,P3&user_id=xxx&limit=6
# Page panier — Cross-sell agrégé des items du panier
# =====
@router.get("/cart")
async def cart_recommendations(items: str, user_id: str = None, limit: int = 6):
    cart_items = items.split(",")
    # 1. Agréger les cross-sell de chaque item du panier
    all_candidates = set()
    for item in cart_items:
        candidates = await redis.get(f"crosssell:{item}")
        if candidates:
            all_candidates.update(candidates)
    all_candidates -= set(cart_items) # Exclure les produits déjà dans le panier

    # 2. Re-rank via MetaRank si user connecté
    candidates_list = list(all_candidates)
    ranking_id = None
    if user_id:
        result = await metarank_client.rank(user_id=user_id, items=candidates_list)
        candidates_list = result["items"]
        ranking_id = result["ranking_id"]

    return {"items": candidates_list[:limit], "type": "cart-cross-sell", "rankingId": ranking_id}

# =====
# GET /api/health
# =====
@router.get("/health")
async def health():
    return {"status": "ok", "metarank": "connected", "redis": "connected"}

```

## 16. Client MetaRank ( services/metarank\_client.py ) :

```

import httpx
from uuid import uuid4
from datetime import datetime

METARANK_URL = "http://localhost:8013"

async def rank(user_id: str, items: list[str], session_id: str = None) -> dict:
    """Appel MetaRank POST /api/v1/rank pour re-ranking personnalisé.
    Retourne {"items": [...], "ranking_id": "rank-xxx"}
    """
    ranking_id = f"rank-{uuid4()}"
    payload = {
        "event": "ranking",
        "id": ranking_id,

```

```

    "user": user_id,
    "session": session_id or f"sess-{user_id}",
    "items": [{ "id": item } for item in items],
    "timestamp": datetime.utcnow().isoformat() + "Z"
}

async with httpx.AsyncClient(timeout=0.5) as client:
    resp = await client.post(f"{METARANK_URL}/api/v1/rank", json=payload)
    result = resp.json()
    return {
        "items": [item["id"] for item in result["items"]],
        "ranking_id": ranking_id
    }

async def send_feedback(event: dict):
    """Envoyer un événement de feedback (interaction) à MetaRank."""
    async with httpx.AsyncClient(timeout=0.5) as client:
        await client.post(f"{METARANK_URL}/api/v1/feedback", json=event)

```

## TRACKING DES ÉVÉNEMENTS UTILISATEUR

### Comment ça fonctionne (flux complet)

User clique sur un produit recommandé

Next.js Frontend

```

POST /occ/v2/lafoirfouille/recommendations/events
{eventType: "click", productCode: "LAMPE-001", rankingId: "rank-abc123"}

```

SAP OCC (RecommendationController)

(HTTP async, fire-and-forget)

FastAPI POST /api/events

PostgreSQL : INSERT INTO user\_events (pour analytics + ré-entraînement)

MetaRank POST /api/v1/feedback (pour re-ranking temps réel)

### Comment lier un ranking à une interaction ? (le **rankingId** )

C'est la clé du tracking closed-loop :

1. Frontend demande des recommandations
 

```

SAP  FastAPI  MetaRank POST /api/v1/rank
MetaRank retourne {items: [...]}
FastAPI génère un rankingId unique  le retourne dans la réponse

```
2. Frontend affiche les produits + stocke rankingId en state React
3. User clique sur un produit dans cette liste de recommandations
 

```

Frontend envoie POST /events {productCode, rankingId: "rank-abc123"}
SAP  FastAPI  MetaRank POST /api/v1/feedback
MetaRank enregistre : "dans le ranking rank-abc123, user a cliqué LAMPE-001"

```

4. Au prochain appel /rank pour ce user, MetaRank tient compte de ce click

## Sem. 8 — Endpoint de tracking FastAPI

### 17. routers/events.py :

```
from pydantic import BaseModel
from uuid import uuid4
from datetime import datetime

class UserEvent(BaseModel):
    user_id: str
    event_type: str # "click", "add_to_cart", "purchase", "wishlist"
    product_code: str
    session_id: str
    ranking_id: str = None # L'ID du ranking MetaRank qui a affiché ce produit
    timestamp: str = None

@router.post("/events")
async def track_event(event: UserEvent):
    ts = event.timestamp or datetime.utcnow().isoformat() + "Z"

    # 1. Stocker dans PostgreSQL (historique + ré-entraînement batch)
    await postgres.execute(
        "INSERT INTO user_events (user_id, event_type, product_code, session_id, timestamp) "
        "VALUES ($1, $2, $3, $4, $5)",
        event.user_id, event.event_type, event.product_code, event.session_id, ts
    )

    # 2. Envoyer à MetaRank (re-ranking temps réel)
    if event.ranking_id:
        metarank_event = {
            "event": "interaction",
            "id": f"int-{uuid4()}",
            "ranking": event.ranking_id,
            "item": event.product_code,
            "user": event.user_id,
            "session": event.session_id,
            "type": event.event_type, # click, add_to_cart, purchase
            "timestamp": ts
        }
        await metarank_client.send_feedback(metarank_event)

    return {"status": "ok"}
```

### Quels événements tracker et quand ?

| Événement   | Quand                                                        | Déclenché par                | Données envoyées                     |
|-------------|--------------------------------------------------------------|------------------------------|--------------------------------------|
| click       | User clique sur un produit dans une liste de recommandations | Frontend (onClick)           | productCode, rankingId               |
| add_to_cart | User ajoute un produit au panier depuis une reco             | Frontend (onAddToCart)       | productCode, rankingId               |
| purchase    | Commande confirmée                                           | Backend SAP (après paiement) | tous les productCodes de la commande |

|          |                     |                       |             |
|----------|---------------------|-----------------------|-------------|
| wishlist | Ajout à la wishlist | Frontend (onWishlist) | productCode |
|----------|---------------------|-----------------------|-------------|

## Tracking côté Frontend (Next.js)

```
// Composant RecommendationSection
function RecommendationSection({ type, products, rankingId }) {
  const handleProductClick = (productCode: string) => {
    // Tracker le click avec le rankingId
    fetch('/occ/v2/lafoirfouille/recommendations/events', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        eventType: 'click',
        productCode,
        rankingId // Lié au ranking MetaRank qui a affiché cette liste
      })
    });
  };

  return (
    <div>
      {products.map(p => (
        <ProductCard
          key={p.code}
          product={p}
          onClick={() => handleProductClick(p.code)}
        />
      ))}
    </div>
  );
}
```

## Phase 4 : Intégration SAP Commerce (Sem. 8-10)

### Fichiers Java à créer

#### 18. Service Layer — lafoirfouilleservices :

Fichier :

bin/custom/lafoirfouille/lafoirfouilleservices/src/fr/lafoirfouille/services/recommendation/RecommendationFoirService.java

```
public interface RecommendationFoirService {
  List<String> getTopSales(String userId, int limit);
  List<String> getCrossSell(String productCode, String userId, int limit);
  List<String> getSimilarProducts(String productCode, String userId, int limit);
  List<String> getCartRecommendations(List<String> cartItems, String userId, int limit);
  void trackEvent(String userId, String sessionId, String eventType,
    String productCode, String rankingId);
}
```

Fichier :

bin/custom/lafoirfouille/lafoirfouilleservices/src/fr/lafoirfouille/services/recommendation/impl/DefaultRecommendationFoirService.j

- Utilise RestTemplate (pattern existant dans WebServicesUtils )
- Appelle FastAPI sur <http://localhost:8000/api/recommendations/...>

- Timeout : 500ms max
- **Fallback** : si FastAPI/MetaRank down `SimpleSuggestionService` existant (package `fr.lafoirfouille.core.suggestion` )
- Config URL dans `local.properties` : `lafoirfouille.recommendation.api.url=http://localhost:8000`
- `trackEvent()` : fire-and-forget async vers FastAPI `POST /api/events`

## 19. Facade Layer — `lafoirfouillefacades` :

Fichier :

`bin/custom/lafoirfouille/lafoirfouillefacades/src/fr/lafoirfouille/facades/recommendation/RecommendationFoirFacade.java`

```
public interface RecommendationFoirFacade {
    RecommendationData getTopSales(String userId, int limit);
    RecommendationData getCrossSell(String productCode, String userId, int limit);
    RecommendationData getSimilarProducts(String productCode, String userId, int limit);
    RecommendationData getCartRecommendations(List<String> cartItems, String userId, int limit);
    void trackEvent(String userId, String sessionId, String eventType,
        String productCode, String rankingId);
}
```

Fichier :

`bin/custom/lafoirfouille/lafoirfouillefacades/src/fr/lafoirfouille/facades/recommendation/impl/DefaultRecommendationFoirFacade.java`

- Reçoit product codes du service
- Convertit en `ProductData` via `productConverter` existant (même pattern que `DefaultSimpleSuggestionFacade` dans `fr.lafoirfouille.facades.suggestion` )
- Retourne `RecommendationData`

DTO :

`bin/custom/lafoirfouille/lafoirfouillefacades/src/fr/lafoirfouille/facades/recommendation/data/RecommendationData.java`

```
public class RecommendationData {
    private List<ProductData> products;
    private String type; // "top-sales", "cross-sell", "similar"
    private boolean personalized;
    private String rankingId; // MetaRank ranking ID pour le tracking frontend
    // + getters/setters
}
```

## 20. OCC Controller — `lafoirfouilleocc` :

Fichier :

`bin/custom/lafoirfouille/lafoirfouilleocc/src/fr/lafoirfouille/webservices/controllers/RecommendationController.java`

- Extends `BaseController` (pattern existant dans le projet)
- Package : `fr.lafoirfouille.webservices.controllers`

```
@RestController
@RequestMapping("/{baseSiteId}/recommendations")
public class RecommendationController extends BaseController
{
    @Autowired
    private RecommendationFoirFacade recommendationFoirFacade;

    // GET /occ/v2/{site}/recommendations/top-sales?limit=12
    @GetMapping("/top-sales")
    public RecommendationWsFoirDTO getTopSales(
        @RequestParam(defaultValue = "12") int limit,
        HttpServletRequest request)
    {
```

```

        String userId = getUserId(request);
        RecommendationData data = recommendationFoirFacade.getTopSales(userId, limit);
        return convertToWsDTO(data);
    }

    // GET /occ/v2/{site}/recommendations/cross-sell/{productCode}?limit=6
    @GetMapping("/cross-sell/{productCode}")
    public RecommendationWsFoirDTO getCrossSell(
        @PathVariable String productCode,
        @RequestParam(defaultValue = "6") int limit,
        HttpServletRequest request)
    {
        String userId = getUserId(request);
        RecommendationData data = recommendationFoirFacade.getCrossSell(productCode, userId, limit);
        return convertToWsDTO(data);
    }

    // GET /occ/v2/{site}/recommendations/similar/{productCode}?limit=8
    @GetMapping("/similar/{productCode}")
    public RecommendationWsFoirDTO getSimilar(
        @PathVariable String productCode,
        @RequestParam(defaultValue = "8") int limit,
        HttpServletRequest request)
    {
        String userId = getUserId(request);
        RecommendationData data = recommendationFoirFacade.getSimilarProducts(productCode, userId, limit);
        return convertToWsDTO(data);
    }

    // GET /occ/v2/{site}/recommendations/cart?items=P1,P2&limit=6
    @GetMapping("/cart")
    public RecommendationWsFoirDTO getCartRecommendations(
        @RequestParam String items,
        @RequestParam(defaultValue = "6") int limit,
        HttpServletRequest request)
    {
        String userId = getUserId(request);
        List<String> cartItems = Arrays.asList(items.split(","));
        RecommendationData data = recommendationFoirFacade.getCartRecommendations(cartItems, userId, limit);
        return convertToWsDTO(data);
    }

    // POST /occ/v2/{site}/recommendations/events
    @PostMapping("/events")
    public ResponseEntity<Void> trackEvent(
        @RequestBody RecommendationEventWsDTO event,
        HttpServletRequest request)
    {
        String userId = getUserId(request);
        String sessionId = getSessionId(request);
        recommendationFoirFacade.trackEvent(
            userId, sessionId,
            event.getEventType(),
            event.getProductCode(),
            event.getRankingId()
        );
        return ResponseEntity.ok().build();
    }

```

```
}
}
```

#### Endpoints OCC récapitulatif :

| Endpoint OCC (appelé par Next.js)                           | Méthode Java             | Paramètres                                |
|-------------------------------------------------------------|--------------------------|-------------------------------------------|
| GET /occ/v2/{site}/recommendations/top-sales                | getTopSales()            | ?limit=12                                 |
| GET /occ/v2/{site}/recommendations/cross-sell/{productCode} | getCrossSell()           | ?limit=6                                  |
| GET /occ/v2/{site}/recommendations/similar/{productCode}    | getSimilar()             | ?limit=8                                  |
| GET /occ/v2/{site}/recommendations/cart                     | getCartRecommendations() | ?items=P1,P2&limit=6                      |
| POST /occ/v2/{site}/recommendations/events                  | trackEvent()             | body: {eventType, productCode, rankingId} |

- Le `userId` est récupéré automatiquement depuis le token d'authentification SAP (user connecté) ou `null` (anonyme)
- Le `rankingId` est propagé dans la réponse pour que le frontend puisse le renvoyer lors des clicks

#### 21. Spring XML beans à ajouter :

lafoirfouilleservices-spring.xml :

```
<bean id="recommendationFoirService"
    class="fr.lafoirfouille.services.recommendation.impl.DefaultRecommendationFoirService">
    <property name="recommendationApiUrl" value="${lafoirfouille.recommendation.api.url}" />
    <property name="restTemplate" ref="restTemplate" />
    <property name="simpleSuggestionService" ref="simpleSuggestionService" />
</bean>
```

lafoirfouillefacades-spring.xml :

```
<bean id="recommendationFoirFacade"
    class="fr.lafoirfouille.facades.recommendation.impl.DefaultRecommendationFoirFacade">
    <property name="recommendationFoirService" ref="recommendationFoirService" />
    <property name="productConverter" ref="productConverter" />
</bean>
```

config/local.properties :

```
lafoirfouille.recommendation.api.url=http://localhost:8000
lafoirfouille.recommendation.api.timeout=500
```

#### 22. WsDTO pour OCC :

bin/custom/lafoirfouille/lafoirfouilleocc/src/fr/lafoirfouille/webservices/data/RecommendationWsFoirDTO.java

```
public class RecommendationWsFoirDTO {
    private List<ProductWsDTO> products;
    private String type;
    private boolean personalized;
    private String rankingId;
    // + getters/setters
}
```

```
bin/custom/lafoirfouille/lafoirfouilleocc/src/fr/lafoirfouille/webservices/data/RecommendationEventWsDTO.java
```

```
public class RecommendationEventWsDTO {  
    private String eventType;  
    private String productCode;  
    private String rankingId;  
    // + getters/setters  
}
```

---

## Phase 5 : Frontend Next.js + Démo (Sem. 10-12)

### Sem. 10-11 — Composants Next.js

#### 23. Appels API depuis Next.js :

```
// Page accueil — Top ventes (personnalisé si connecté, le backend gère)  
GET /occ/v2/lafoirfouille/recommendations/top-sales?limit=12  
  
// Page produit — 2 blocs de recommandations  
GET /occ/v2/lafoirfouille/recommendations/similar/LAMPE-001?limit=8  
GET /occ/v2/lafoirfouille/recommendations/cross-sell/LAMPE-001?limit=6  
  
// Page panier — Cross-sell agrégé  
GET /occ/v2/lafoirfouille/recommendations/cart?items=P1,P2,P3&limit=6
```

#### 24. Composant `<RecommendationSection />` :

- Lazy loading + skeleton loading
- Stocke le `rankingId` de la réponse en state React
- onClick sur un produit → POST tracking event avec `rankingId`
- Affiche un titre contextuel (“Nos recommandations”, “Produits similaires”, etc.)

#### 25. Placements frontend :

- **Page accueil (connecté)** : “Nos recommandations pour vous” `type=top-sales`
- **Page accueil (anonyme)** : “Meilleures ventes” `type=top-sales` (pas de re-rank)
- **Page produit** : “Produits similaires” + “Souvent achetés ensemble”
- **Page panier** : “Complétez votre commande”

### Sem. 12 — Finalisation

#### 26. Tests :

- Unit tests FastAPI (pytest) : chaque endpoint, client MetaRank, tracking
- Test connexion MetaRank : `POST /api/v1/rank` avec données de test
- Test flux complet : Next.js → SAP OCC → FastAPI → Redis + MetaRank
- Évaluation finale : notebook `evaluation.ipynb` avec graphiques comparatifs

#### 27. Documentation :

- README microservice (installation PostgreSQL/Redis/MetaRank, lancement, endpoints)
- Schéma architecture (diagramme complet)
- Rapport de stage : contexte, choix techniques, résultats métriques, limites, perspectives

---

## Redis Cache Strategy

| Clé Redis                | Contenu                           | TTL | Calculé par                        |
|--------------------------|-----------------------------------|-----|------------------------------------|
| topsales:global          | Top 50 global (time-decay SQL)    | 1h  | precompute_cache.py (cron 1x/jour) |
| crosssell:{product_code} | Top 20 cross-sell (LightFM)       | 24h | precompute_cache.py                |
| similar:{product_code}   | Top 20 similaires (Sentence-BERT) | 24h | precompute_cache.py                |

**PAS de cache par user** — la personnalisation est faite en temps réel par MetaRank (~10ms).

## Fichiers à créer

### Microservice Python ( recommendation-engine/ )

- main.py , config.py
- routers/recommendations.py , routers/events.py
- services/metarank\_client.py , services/candidate\_service.py , services/tracking\_service.py
- data/postgres\_client.py , data/redis\_client.py
- scripts/convert\_orders\_to\_events.py , scripts/export\_sap\_data.py , scripts/load\_csv\_to\_postgres.py
- scripts/train\_lightfm.py , scripts/compute\_embeddings.py , scripts/precompute\_cache.py
- metarank/config.yml
- notebooks/eda.ipynb , notebooks/evaluation.ipynb
- tests/test\_recommendations.py
- requirements.txt , README.md

### SAP Commerce (Java)

- lafoirfouilleservices/.../recommendation/RecommendationFoirService.java (interface)
- lafoirfouilleservices/.../recommendation/impl/DefaultRecommendationFoirService.java
- lafoirfouillefacades/.../recommendation/RecommendationFoirFacade.java (interface)
- lafoirfouillefacades/.../recommendation/impl/DefaultRecommendationFoirFacade.java
- lafoirfouillefacades/.../recommendation/data/RecommendationData.java (DTO)
- lafoirfouilleocc/.../controllers/RecommendationController.java
- lafoirfouilleocc/.../data/RecommendationWsFoirDTO.java
- lafoirfouilleocc/.../data/RecommendationEventWsDTO.java

### À modifier

- lafoirfouilleservices-spring.xml — ajouter bean recommendationFoirService
- lafoirfouillefacades-spring.xml — ajouter bean recommendationFoirFacade
- config/local.properties — ajouter URL FastAPI

## Planning (12 semaines, 1 stagiaire solo)

| Sem. | Tâche                                                         | Livrable                          |
|------|---------------------------------------------------------------|-----------------------------------|
| 1    | Setup PostgreSQL/Redis/MetaRank + export MSSQL CSV PostgreSQL | Infra prête                       |
| 2    | Convertir 189k commandes events MetaRank JSONL + PostgreSQL   | ~1M events prêts                  |
| 3    | Config MetaRank + entraîner + EDA notebook                    | MetaRank opérationnel             |
| 4    | SQL time-decay top ventes + Sentence-BERT embeddings Redis    | Candidats top ventes + similaires |
| 5    | LightFM cross-sell + pré-calcul Redis                         | Candidats cross-sell              |
| 6    | Évaluation offline + pipeline batch (precompute_cache.py)     | Métriques + cron                  |
| 7    | FastAPI endpoints recommandations + client MetaRank           | API fonctionnelle                 |

|    |                                                          |                         |
|----|----------------------------------------------------------|-------------------------|
| 8  | Tracking events (FastAPI + début integ SAP service Java) | Tracking opérationnel   |
| 9  | Intégration SAP (facade + controller OCC)                | Endpoints OCC testables |
| 10 | Tracking SAP FastAPI MetaRank + début frontend Next.js   | Tracking live           |
| 11 | Composants Next.js + tests                               | Prototype intégré       |
| 12 | Tests finaux, documentation, rapport de stage            | Livrable complet        |

## Décisions techniques

1. **MetaRank** pour le re-ranking personnalisé temps réel (~10ms) — remplace le scoring hybride manuel
2. **LightFM (WARP)** conservé pour la génération de candidats cross-sell — compétence ML à présenter en soutenance
3. **Sentence-BERT (all-MiniLM-L6-v2)** pour la similarité sémantique produit
4. **SQL time-decay** pour les top ventes (pas de ML nécessaire, cron 1x/jour)
5. **Pas de simulation Selenium** — conversion directe des commandes historiques en events MetaRank JSONL
6. **rankingId** pour lier les impressions aux clicks (closed-loop tracking)
7. **SAP comme proxy** frontend ne parle jamais directement à Python/MetaRank
8. **Fallback SimpleSuggestionService** si FastAPI/MetaRank down, SAP renvoie les suggestions basiques existantes
9. **Partie Java assistée par l'équipe** le stagiaire se concentre sur Python/ML
10. **Pas de cache par user dans Redis** MetaRank fait la personnalisation en temps réel